Docker Commands Cheat Sheet

DEVOPS-006: Docker Fundamentals & Containerization | Task 10 | Time: 30 Minutes

`docker --version`

PURPOSE

Displays the currently installed Docker version. Useful for confirming Docker is installed and checking compatibility before running commands.

EXAMPLE

```
docker --version
```

EXPECTED RESULT

Docker version 24.0.7, build afdd53b

`docker pull`

PURPOSE

Downloads an image from a registry (like Docker Hub) onto the local machine, without creating or running a container from it.

EXAMPLE

```
docker pull nginx
```

EXPECTED RESULT

Docker downloads the nginx image layers and confirms with a message like "Status: Downloaded newer image for nginx:latest".

`docker images`

PURPOSE

Lists all Docker images currently stored on the local machine, along with their tag, image ID, creation date, and size.

EXAMPLE

```
docker images
```

EXPECTED RESULT

A table showing columns REPOSITORY, TAG, IMAGE ID, CREATED, and SIZE for each downloaded image.

`docker run`

PURPOSE

Creates and starts a new container from an image in one step. Most commonly used flag is -d for detached (background) mode.

EXAMPLE

```
docker run -d --name web-server nginx
```

EXPECTED RESULT

A new container is created from the nginx image, started immediately, and its container ID is printed. docker ps will show it running.

`docker ps`

PURPOSE

Lists containers that are currently running.

EXAMPLE

```
docker ps
```

EXPECTED RESULT

A table with CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, PORTS, and NAMES — only for active containers.

`docker ps -a`

PURPOSE

Lists all containers, regardless of state — running, stopped, or exited.

EXAMPLE

```
docker ps -a
```

EXPECTED RESULT

Same table format as docker ps, but also includes stopped/exited containers with a STATUS like "Exited (0) 2 minutes ago".

`docker start`

PURPOSE

Starts an existing container that was previously stopped (or created but never started).

EXAMPLE

```
docker start web-server
```

EXPECTED RESULT

The container transitions to a running state, and its name/ID is printed. docker ps will now list it.

`docker stop`

PURPOSE

Gracefully stops a running container by sending a SIGTERM signal (and SIGKILL after a timeout if it doesn't stop in time).

EXAMPLE

```
docker stop web-server
```

EXPECTED RESULT

The container shuts down cleanly, and its name is printed as confirmation. It no longer appears in docker ps, but still shows in docker ps -a.

`docker restart`

PURPOSE

Stops and then immediately starts a container again — useful for applying config reloads or recovering a stuck container without removing it.

EXAMPLE

```
docker restart web-server
```

EXPECTED RESULT

A brief pause while the container stops and restarts, then it shows as running again in docker ps.

`docker rm`

PURPOSE

Permanently deletes a container. The container must be stopped first, unless the -f (force) flag is used.

EXAMPLE

```
docker rm -f web-server
```

EXPECTED RESULT

The container is removed and its name is printed. It no longer appears in docker ps -a.

`docker rmi`

PURPOSE

Removes a Docker image from local storage (not a container). Fails if a container still depends on that image, unless -f is used.

EXAMPLE

```
docker rmi nginx
```

EXPECTED RESULT

The image is deleted, and it no longer appears in docker images.

`docker logs`

PURPOSE

Displays the stdout/stderr output generated by a container — useful for debugging or monitoring what's happening inside it.

EXAMPLE

```
docker logs web-server
```

EXPECTED RESULT

Printed log output from the container (e.g. Nginx startup messages, request logs). Add -f to follow the logs live.

`docker inspect`

PURPOSE

Returns detailed low-level metadata about a container or image in JSON format — including ID, status, network settings, mounts, and creation time.

EXAMPLE

```
docker inspect web-server
```

EXPECTED RESULT

A large JSON object containing full configuration and runtime details of the container.

`docker exec`

PURPOSE

Runs a command inside an already-running container, without stopping or restarting it. Commonly used to open an interactive shell for debugging.

EXAMPLE

```
docker exec -it web-server /bin/bash
```

EXPECTED RESULT

Drops into an interactive Bash shell inside the running web-server container, letting you inspect files or processes directly.

The trainee should be able to explain the purpose of every command above.